

Informe Proyecto Módulo 6: BookList Sistema de Gestión de Libros

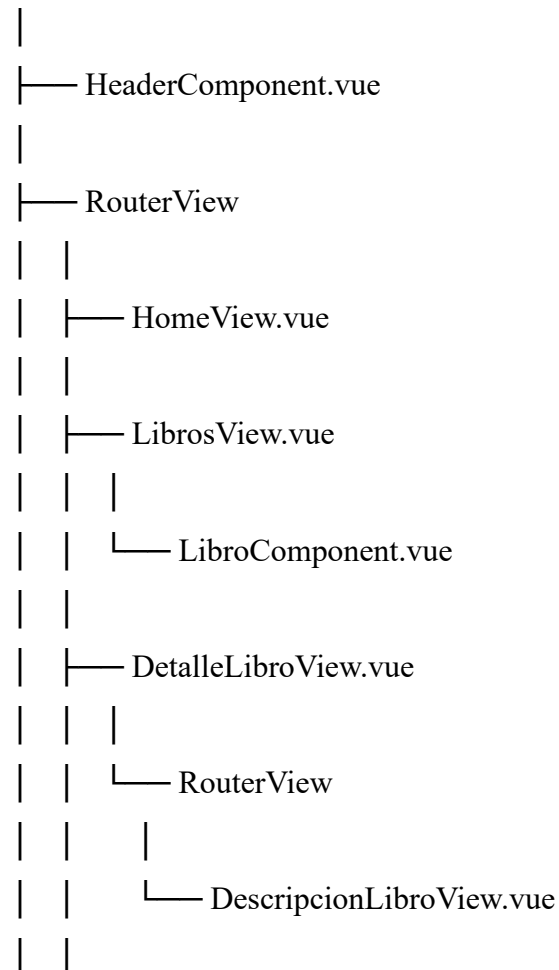
1. Descripción

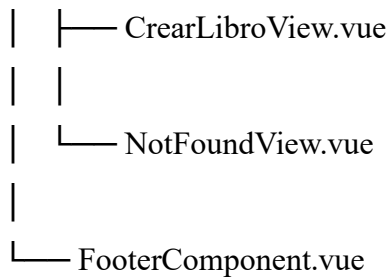
BookList es una aplicación SPA desarrollada utilizando Vue 3 y Vue Router, que permite a un usuario acceder mediante inicio de sesión o continuar como invitado, visualizar un catálogo de libros, filtrar por autor y/o por categoría de libro, consultar el detalle de cada libro, acceder a su descripción mediante una ruta anidada, agregar nuevos libros y eliminar registros existentes. La aplicación también contempla casos excepcionales de navegación, diferenciando entre un ID de libro inexistente dentro de una ruta válida y una ruta no definida, la cual es manejada mediante una vista de error 404.

El proyecto fue desarrollado aplicando los conceptos vistos durante el módulo 6, y priorizando la reutilización de componentes, la separación de responsabilidades, el manejo reactivo de datos y una interfaz consistente para el usuario.

2. Arquitectura de la aplicación

App.vue





- App contiene la estructura principal y administra el estado compartido de la aplicación.
- Vue Router administra la navegación entre las distintas vistas.
- HeaderComponent corresponde a la barra superior con navegación e identificación del usuario activo, diferenciando entre usuario registrado e invitado.
- FooterComponent corresponde al pie de página común de la aplicación.
- HeaderComponent se muestra en las vistas internas de la aplicación, excepto en HomeView y NotFoundView.
- El RouterView principal permite cargar la vista correspondiente a la ruta actual. En App.vue se utiliza RouterView con v-slot y `<component :is="Component">` para controlar el renderizado de las vistas y asignar las props, eventos o KeepAlive que corresponden en cada caso.
- DetalleLibroView contiene un RouterView anidado con v-slot y `<component :is="Component">`, encargado de renderizar DescripcionLibroView cuando el usuario accede a la ruta hija de descripción (al hacer click en el botón de Ver descripción) y para pasarle mediante props el objeto libro correspondiente.

3. Vistas implementadas

Se opta por la implementación de 6 vistas (carpeta views), cuyo contenido, en particular, se detalla a continuación:

a) HomeView

- Formulario de inicio de sesión para usuarios registrados.
- Validación de credenciales y mensajes de error correspondientes.
- Opción para cambiar entre el formulario de inicio de sesión y el formulario de ingreso como invitado.
- Formulario de ingreso como invitado mediante nombre de usuario.
- Validación del nombre de invitado.
- Mensaje de ingreso exitoso para usuario registrado e invitado.
- Los mensajes de error se limpian cuando el usuario comienza a corregir o completar los campos faltantes del formulario, tanto en inicio de sesión como en el ingreso como

invitado. Para esto se utiliza `@input`, asociado a una función que elimina los mensajes de error.

- Link para redireccionar al catálogo de libros después de un ingreso exitoso.
- Contador reactivo con opciones para incrementar, decrementar y resetear su valor. Los botones Decrementar y Resetear sólo se muestran cuando el contador es distinto de cero, y el contador no puede disminuir a valores negativos.
- Se utiliza `KeepAlive` para conservar el estado local de la vista Home al navegar hacia otras vistas de la aplicación. De esta manera, el valor del contador reactivo se mantiene cuando el usuario regresa a Home.

b) LibrosView

- Listado de libros (o catálogo). Cada libro contiene el título, autor y categoría.
- Filtros por autor y categoría generados a partir de los libros existentes.
- Los filtros se muestran mediante `v-show` solamente cuando existen libros registrados.
- Mensaje cuando existen libros, pero los filtros seleccionados no producen resultados válidos.
- Mensaje diferente cuando el catálogo de libros se encuentra completamente vacío.
- Cada libro es representado mediante `LibroComponent.vue`, desde donde es posible acceder a su detalle o eliminarlo.

c) DetalleLibroView

- Recibe el id del libro como prop desde el parámetro dinámico de la ruta `/libros/:id`, mediante la configuración `props:true` de Vue Router.
- Recibe, además, desde `App.vue`, el arreglo actualizado de libros.
- Utiliza una propiedad computada (`computed`) para buscar dentro del arreglo el libro cuyo id corresponda al parámetro recibido en la ruta.
- El título, autor y categoría mostrados en el detalle se obtienen directamente del libro encontrado en el arreglo (no mediante parámetros query).
- Caso especial: Si el usuario manipula la dirección, y el ID ingresado no corresponde actualmente a ningún libro registrado, muestra el mensaje de “Libro no encontrado” y un enlace para volver al inicio.
- Cuando el libro existe, permite acceder a su descripción mediante la ruta anidada o hija `/libros/:id/descripcion`.

- Utiliza `useRoute()` para identificar si la ruta actual corresponde al detalle principal o a la ruta hija de descripción y mostrar el enlace de “Ver descripción” u “Ocultar descripción” según corresponda (como botón).
- Incluye un `RouterView` anidado donde se renderiza `DescripcionLibroView`.

d) DescripcionLibroView

- Corresponde a una vista hija de `DetalleLibroView`.
- Se renderiza mediante el `RouterView` anidado cuando la ruta actual es `/libros/:id/descripcion`.
- Recibe mediante props el objeto libro encontrado previamente en `DetalleLibroView`.
- Muestra la descripción registrada del libro.
- Al utilizar el mismo objeto libro obtenido desde el arreglo principal, la descripción permanece asociada al mismo registro identificado por el id de la ruta.

e) CrearLibroView

- Formulario para añadir un nuevo libro mediante campos de título, categoría, autor y descripción.
- Se utiliza `@submit.prevent` para controlar el envío del formulario mediante `handleSubmit()` y prevenir el comportamiento por defecto del navegador de recargar la página.
- Se utiliza `@keydown.enter.prevent` en los campos Título y Autor para permitir ejecutar el registro del libro mediante la tecla Enter, llamando a `handleSubmit()`. Si todos los campos del formulario están completos (son todos obligatorios), el libro se registra, en caso contrario, se muestran los mensajes de validación correspondientes.
- Los cuatro campos del formulario son obligatorios.
- Se realizan validaciones antes de emitir el nuevo libro hacia `App.vue`.
- Si se intenta guardar un libro con campos faltantes, se muestra un mensaje de error. Este mensaje se limpia cuando el usuario comienza a corregir y completar los datos faltantes, evitando mantener visible un error que ya está siendo solucionado, para lo cual se utilizan `@input` y `@change` en los campos del formulario, asociados a una respectiva función que elimina dicho mensaje.
- Se incluye una vista previa que refleja en tiempo real los datos ingresados mediante `v-model`.
- Se incluye una ayuda contextual mediante botón informativo que utiliza el modificador `.once`. Al activarlo, se muestran indicaciones para completar correctamente los campos.

obligatorios del formulario y registrar el libro (botón amarillo en la esquina superior derecha).

- Después de registrar correctamente un libro, se limpia el formulario y se muestra un mensaje de éxito y un enlace para acceder al listado de libros.

f) NotFoundView

- Vista destinada exclusivamente al manejo de rutas inexistentes.
- Se activa mediante la ruta comodín `/:pathMatch(.*)*` configurada en Vue Router.
- Muestra el mensaje de “Error 404: Pagina no encontrada”.
- Informa que no fue posible encontrar la ruta especificada.
- Incluye un enlace para volver al inicio.
- Este caso se diferencia del mensaje de “Libro no encontrado” de `DetalleLibroView`, ya que en ese caso la ruta `/libros/:id` sí existe, pero el id solicitado no corresponde a un libro registrado.

4. Decisiones de desarrollo

Se utiliza Composition API para administrar el estado reactivo de la aplicación. El estado principal se mantiene en `App.vue`, donde se encuentran los arreglos reactivos de libros y usuarios, el usuario activo, la variable utilizada para generar identificadores consecutivos y las funciones principales para agregar y eliminar libros, e iniciar la sesión del usuario activo (usuario o invitado). `HomeView` gestiona tanto la validación del usuario registrado como la creación del usuario invitado y comunica el usuario resultante hacia `App.vue` mediante el evento personalizado `login-exitoso`. Por lo tanto, los datos se distribuyen hacia las vistas mediante props, mientras que las acciones realizadas por los componentes hijos se comunican hacia los componentes padres mediante eventos personalizados.

El contador reactivo se mantiene como estado local de `HomeView.vue`. Para conservar su valor al navegar hacia otras vistas y posteriormente regresar a Home, se utiliza el componente `KeepAlive` en `App.vue`, aplicado únicamente a la vista `HomeView`. De esta forma, se conserva la instancia y el estado local de esta vista mientras el usuario navega por la aplicación, sin necesidad de trasladar el contador al estado principal de `App.vue`. El usuario activo, en cambio, se administra desde `App.vue`, ya que corresponde a un estado compartido entre distintas vistas y componentes de la aplicación.

En lo que respecta a la componentización, se creó `LibroComponent.vue`, que permite reutilizar el mismo diseño de tarjeta para mostrar cada libro en el catálogo (Tarjeta reutilizable). La comunicación entre componentes se realiza mediante props y eventos personalizados. Por ejemplo, `App.vue` entrega el arreglo `libros` a `LibrosView.vue`; posteriormente, cada libro es enviado mediante una prop a `LibroComponent.vue`. Cuando se elimina un libro, `LibroComponent` emite el evento personalizado `eliminar-libro`, que es propagado por `LibrosView.vue` hasta `App.vue`, donde, finalmente, se ejecuta la eliminación.

Para mantener una única fuente de datos para los libros, se decidió que LibroComponent.vue envíe únicamente el id del libro en la ruta al acceder al detalle. Los datos del libro no se transportan mediante query params. DetalleLibroView recibe el arreglo libros desde App.vue y utiliza el id recibido mediante la ruta dinámica para localizar el objeto correspondiente con una propiedad computada. Esta decisión evita que el id de la ruta y los datos mostrados puedan representar libros diferentes y permite que el detalle utilice siempre la información existente actualmente en el estado principal de la aplicación.

Respecto a la navegación, se tiene que ésta se configura mediante Vue Router. Para el detalle de cada libro, se utiliza la ruta dinámica /libros/:id, configurada con props:true, permitiendo que el parámetro id sea recibido directamente como prop por DetalleLibroView.vue. Además, se incorporó una ruta hija para mostrar la descripción del libro y una ruta comodín para manejar rutas inexistentes. Rutas:

- / (ir a HomeView.vue)
- /libros (ir a LibrosView.vue)
- /libros/:id (ir a DetalleLibroView.vue)
- /libros/:id/descripcion (ir a DescripcionLibroView.vue, como ruta hija de DetalleLibroView.vue)
- /crear (ir a CrearLibroView.vue)
- /* (ir a NotFoundView.vue)

Se diferencian dos casos excepcionales de navegación:

a) Libro no encontrado: La ruta /libros/:id es válida, pero el id ingresado no corresponde actualmente a ningún libro del arreglo. Este caso es manejado directamente por DetalleLibroView.

b) Página no encontrada: La dirección ingresada no coincide con ninguna ruta definida en la aplicación. Vue Router captura este caso mediante la ruta comodín y renderiza NotFoundView con un error 404.

Adicionalmente, durante el despliegue de la aplicación en GitHub Pages, se detectó que, al recargar una ruta interna de la SPA, GitHub Pages intentaba localizar dicha ruta como un recurso físico y mostraba su propio error 404. Para solucionar este comportamiento, siguiendo recomendaciones de IA (ChatGPT y Copilot), se creó un archivo 404.html que guarda temporalmente la ruta solicitada mediante sessionStorage y redirige al inicio de la aplicación. A su vez, se incorporó un script en index.html que recupera dicha ruta y la restaura mediante history.replaceState(), permitiendo que VueRouter continúe gestionando la navegación dentro de la SPA.

Los filtros por autor y categoría se implementan mediante propiedades computadas (computed). También se generan dinámicamente las opciones disponibles de autores y categorías a partir del arreglo de libros, evitando valores repetidos. Una tercera propiedad computada obtiene los libros que cumplen simultáneamente con los filtros seleccionados. Adicionalmente, se opta por mostrar mensajes distintos para catálogo vacío y búsqueda por filtros sin resultados.

En lo que respecta al punto de agregar un nuevo libro a la lista o catálogo de libros, se decide incorporar una vista previa del libro antes de registrarlo, que muestra en tiempo real lo que va ingresando el usuario en la página. Al agregar un nuevo libro, su identificador se genera automáticamente a partir del último ID existente, utilizando la variable reactiva `siguienteId`. De esta manera, el usuario no necesita ingresar manualmente un identificador y cada nuevo registro obtiene un id consecutivo. Además, se incorpora un botón informativo de ayuda contextual que utiliza el modificador `.once` y permite mostrar indicaciones para completar correctamente el formulario. Cuando el registro se realiza correctamente, los campos son limpiados y se muestra una confirmación junto con un enlace al listado de libros, permitiendo comprobar que el nuevo libro fue incorporado al catálogo.

Por otra parte, se decide que el usuario activo (logueado o invitado) permanezca visible en el encabezado en todas las distintas vistas de la aplicación mediante `HeaderComponent`. El encabezado diferencia si el acceso corresponde a un usuario registrado o a un invitado, mostrando el nombre correspondiente.

5. Funcionalidades implementadas

- Inicio de sesión usuario registrado.
- Validación de credenciales.
- Ingreso como invitado mediante nombre de usuario.
- Validación del ingreso como invitado.
- Limpieza automática de los mensajes de error al comenzar a corregir los campos de inicio de sesión o ingreso como invitado.
- Identificación del usuario activo en el encabezado, diferenciando entre usuario registrado e invitado.
- Contador reactivo cuyo estado se conserva mediante `KeepAlive`.
- Listado reactivo de libros.
- Filtros por autor y categoría.
- Mensaje para filtros sin resultados.
- Mensaje específico para catálogo vacío.
- Detalle del libro mediante la ruta dinámica `/libros/:id`.
- Validación de ID de libro inexistente.
- Acceso a la descripción mediante ruta anidada `/libros/:id/descripcion`.
- Mostrar y ocultar la descripción mediante navegación entre la ruta padre y su ruta hija.
- Formulario para agregar libros.
- Validación del título, categoría, autor y descripción como campos obligatorios.

- Limpieza del mensaje de error cuando el usuario comienza a corregir y completar el formulario.
- Vista previa del libro en tiempo real.
- Prevención del comportamiento predeterminado del formulario mediante `.prevent`.
- Uso de `.once` en la ayuda contextual.
- Mensaje de ayuda contextual para completar correctamente el formulario de registro.
- Limpieza del formulario y visualización de mensaje de éxito con acceso directo al listado después de registrar exitosamente un libro.
- Generación automática de ID consecutivo para nuevos libros.
- Manejo de rutas inexistentes mediante `NotFoundView`.
- Diferenciación entre libro no encontrado y ruta inexistente.
- Navegación SPA.

6. Conceptos Vue utilizados

- Composition API
- `ref()`
- `reactive()`
- `computed()`
- Vue Router
- `createRouter()`
- `createWebHistory()`
- Rutas dinámicas o paramétricas.
- Rutas anidadas (`children`)
- Ruta comodín para manejo de error 404
- `RouterLink`
- `RouterView` normal, anidado y con `v-slot`.
- Renderizado dinámico de componentes mediante `<component :is>`
- `useRouter()`
- `useRoute()`
- `KeepAlive`

- Props y defineProps()
- Eventos personalizados con emit y defineEmits()
- v-model
- v-bind
- v-for
- v-show
- v-if
- v-else-if
- v-else
- Modificadores de eventos .prevent y .once
- Eventos @click, @submit, @input, @change, @keydown y personalizados.

7. Dificultades encontradas

- Organización del estado principal en App.vue y comunicación de datos entre vistas y componentes mediante props y eventos personalizados.
- Separación de responsabilidades entre componentes, vistas y estado principal.
- Implementación de filtros reactivos mediante propiedades computadas.
- Implementación de rutas dinámicas y recepción del parámetro id como prop.
- Mantener una única fuente de datos para el detalle del libro, utilizando el id de la ruta para buscar el libro dentro del arreglo principal.
- Implementación de una ruta anidada para mostrar la descripción del libro mediante DescripcionLibroView.
- Comunicación de datos desde DetalleLibroView hacia la vista hija DescripcionLibroView mediante props.
- Diferenciación entre una ruta válida con un ID de libro inexistente y una ruta completamente inexistente manejada mediante NotFoundView.
- Configuración de ruta comodín para el manejo de errores 404.
- Control del comportamiento de los formularios y sus eventos para evitar envíos no deseados y mantener mensajes de validación coherentes con las acciones del usuario.
- Manejo de los distintos estados de acceso de la aplicación, permitiendo diferenciar entre usuario registrado e invitado.

- Comprender e implementar el uso de RouterView con v-slot y <component :is="Component"> para controlar el renderizado de las distintas vistas y poder asignar a cada una las props, eventos o KeepAlive que requería.
- Lograr diseño responsive, coherente y cohesivo.
- Adaptación del funcionamiento de las rutas SPA al despliegue en GitHub Pages, evitando que al recargar una ruta interna se mostrara el error 404 propio de GitHub Pages. En este punto fue necesario seguir instrucciones de la IA.

8. Observaciones adicionales

- Los logos de BookList utilizados en la aplicación fueron diseñados utilizando ChatGPT, siguiendo el estilo visual del proyecto.
- Para realizar la demostración del funcionamiento de la aplicación, se tomaron screenshots de las pantallas principales y de distintos flujos de funcionamiento, que luego se unieron y organizaron en maquetas utilizando ChatGPT. Estas imágenes pueden revisarse en la carpeta src/assets/maqueta.
- Se añade un README_BookList.txt explicando el funcionamiento de la aplicación, que es distinto al README.md generado por defecto con el proyecto. En este último se incorpora un enlace directo al README_BookList.txt para acceder a la documentación completa.
- Se añade un archivo descripciones_libros.txt con datos de tres libros adicionales para hacer pruebas, si así se desea.

9. Demostración del funcionamiento de la aplicación (maqueta)

Maqueta principal

BookList - Pantallas principales

1. Inicio / Login



Bienvenido a BookList
Inicie sesión o registre como invitado para acceder al catálogo de libros.

Inicio de sesión

Correo:

Contraseña:

¿No tiene una cuenta? Continuar como invitado

Continuar invitado

0

Cantidad de libros registrados

© 2026 BookList. Sistema de Gestión de Libros.

2. Inicio / Invitado



Bienvenido a BookList
Inicie sesión o registre como invitado para acceder al catálogo de libros.

Iniciar como invitado

Nombre:

¿Ya tiene una cuenta? Iniciar al inicio de sesión

Continuar invitado

0

Cantidad de libros registrados

© 2026 BookList. Sistema de Gestión de Libros.

3. Detalle del Libro (Usuario registrado)



BookList

Hola, Ximena Cárdenas

[Home](#) [Libros](#) [Agregar libro](#)

Detalle del libro
Información registrada del libro seleccionado.



Meditaciones

Autor: Marco Aurelio

Categoría: Filosofía

[Volver al listado](#)

© 2026 BookList. Sistema de Gestión de Libros.

4. Detalle del Libro (Invitado)



BookList

Hola invitado, María

[Home](#) [Libros](#) [Agregar libro](#)

Detalle del libro
Información registrada del libro seleccionado.



Edipo Rey

Autor: Sófocles

Categoría: Tragedia clásica

[Volver al listado](#)

© 2026 BookList. Sistema de Gestión de Libros.

5. Listado de libros (Usuario registrado)



BookList

Hola, Ximena Cárdenas

[Home](#) [Libros](#) [Agregar libro](#)

Listado de libros
Consulte los libros registrados y acceda a sus detalles.

Selección personalizada con filtros

Autor: Categoría:



Meditaciones

Autor: Marco Aurelio

Categoría: Filosofía



Orgullo y Prejuicio

Autor: Jane Austen

Categoría: Novela



Inés del Alma Mia

Autor: Isabel Allende

Categoría: Novela



La casa de los espíritus

Autor: Isabel Allende

Categoría: Novela

© 2026 BookList. Sistema de Gestión de Libros.

6. Listado de libros (Invitado)



BookList

Hola invitado, María

[Home](#) [Libros](#) [Agregar libro](#)

Listado de libros
Consulte los libros registrados y acceda a sus detalles.

Selección personalizada con filtros

Autor: Categoría:



Meditaciones

Autor: Marco Aurelio

Categoría: Filosofía



Orgullo y Prejuicio

Autor: Jane Austen

Categoría: Novela



Inés del Alma Mia

Autor: Isabel Allende

Categoría: Novela



La casa de los espíritus

Autor: Isabel Allende

Categoría: Novela

© 2026 BookList. Sistema de Gestión de Libros.

7. Agregar libro (Usuario registrado)



BookList

Hola, Ximena Cárdenas

[Home](#) [Libros](#) [Agregar libro](#)

Agregar un libro
Complete el formulario para registrar un nuevo libro.

Datos del libro

Título:

Categoría:

Autor:

Descripción:

Vista previa

Título: Sin título

Categoría: Sin categoría

Autor: Sin autor

Descripción: Sin descripción...

© 2026 BookList. Sistema de Gestión de Libros.

8. Agregar libro (Invitado)



BookList

Hola invitado, María

[Home](#) [Libros](#) [Agregar libro](#)

Agregar un libro
Complete el formulario para registrar un nuevo libro.

Datos del libro

Título:

Categoría:

Autor:

Descripción:

Vista previa

Título: Sin título

Categoría: Sin categoría

Autor: Sin autor

Descripción: Sin descripción...

© 2026 BookList. Sistema de Gestión de Libros.

Maqueta de flujos I

BookList - Flujos de ingreso de usuario e invitado

1. Login - Campos faltantes

Bienvenido a BookList
Inicie sesión o continúe como invitado para acceder al catálogo de libros.

Inicio de sesión

Correo

Contraseña

[¿No tiene una cuenta? Continuar como invitado](#)

Contador reactivo

0

Cantidad de clics realizados

Debe ingresar su correo y contraseña.

© 2026, BookList.
Sistema de Gestión de Libros.

2. Login - Credenciales incorrectas

Bienvenido a BookList
Inicie sesión o continúe como invitado para acceder al catálogo de libros.

Inicio de sesión

Correo

Contraseña

[¿No tiene una cuenta? Continuar como invitado](#)

Contador reactivo

0

Cantidad de clics realizados

El usuario ingresado no existe o la contraseña es incorrecta.

© 2026, BookList.
Sistema de Gestión de Libros.

3. Login - Ingreso exitoso

Bienvenido a BookList
Inicie sesión o continúe como invitado para acceder al catálogo de libros.

Inicio de sesión

Correo

Contraseña

[¿No tiene una cuenta? Continuar como invitado](#)

Contador reactivo

6

Cantidad de clics realizados

Inicio de sesión exitoso.
Sesión iniciada por: **Ximena Ciceros**
[Ir a la lista de libros](#)

© 2026, BookList.
Sistema de Gestión de Libros.

4. Invitado - Campo faltante

Bienvenido a BookList
Inicie sesión o continúe como invitado para acceder al catálogo de libros.

Ingreso como invitado

Nombre

[¿Ya tiene una cuenta? Volver al inicio de sesión](#)

Contador reactivo

0

Cantidad de clics realizados

Debe ingresar su nombre para poder continuar como invitado.

© 2026, BookList.
Sistema de Gestión de Libros.

5. Invitado - Ingreso exitoso

Bienvenido a BookList
Inicie sesión o continúe como invitado para acceder al catálogo de libros.

Ingreso como invitado

Nombre

Contador reactivo

7

Cantidad de clics realizados

Ingreso exitoso como invitado.
Sesión iniciada por: **Maria**
[Ir a la lista de libros](#)

© 2026, BookList.
Sistema de Gestión de Libros.

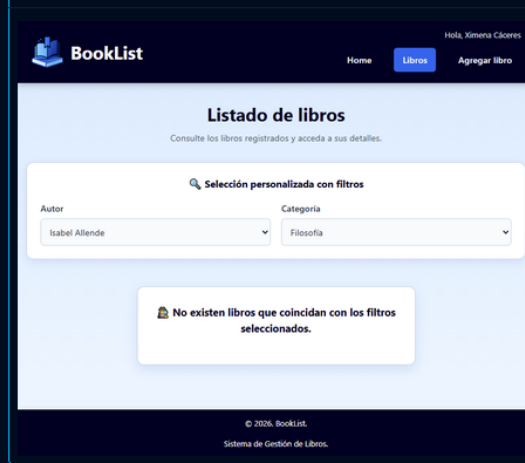
Maqueta de flujos II

BookList - Flujos del listado y filtrado de libros

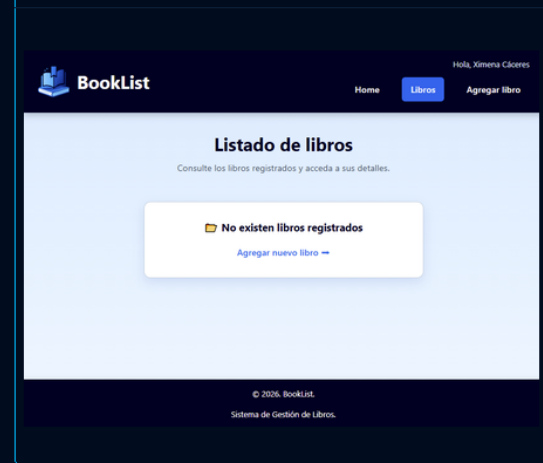
1. Usuario - Filtros con resultados



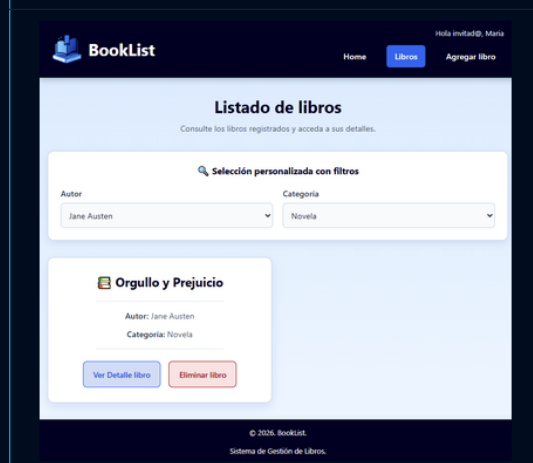
2. Usuario - Filtros sin resultados



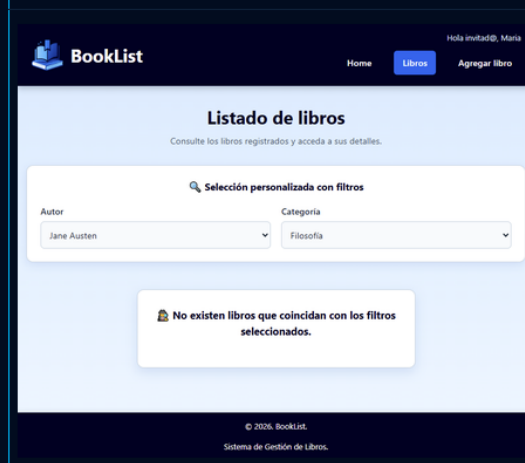
3. Usuario - Sin libros registrados



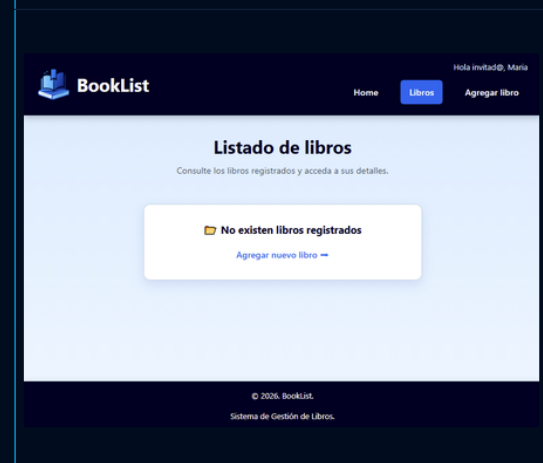
4. Invitado - Filtros con resultados



5. Invitado - Filtros sin resultados



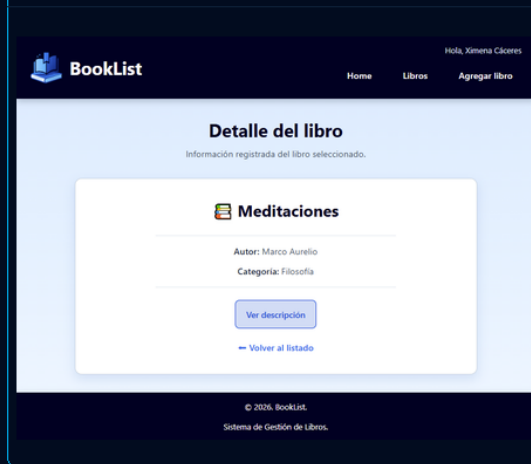
6. Invitado - Sin libros registrados



Maqueta de flujos III

BookList - Flujos del detalle del libro y casos excepcionales

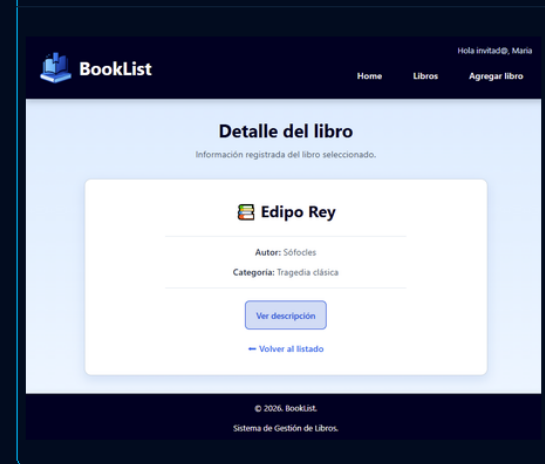
1. Usuario - Descripción oculta



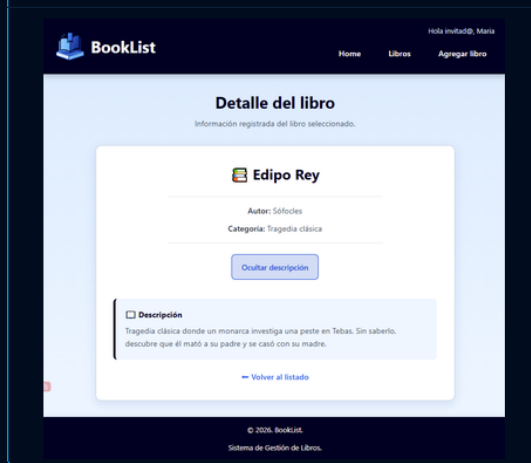
2. Usuario - Descripción mostrada



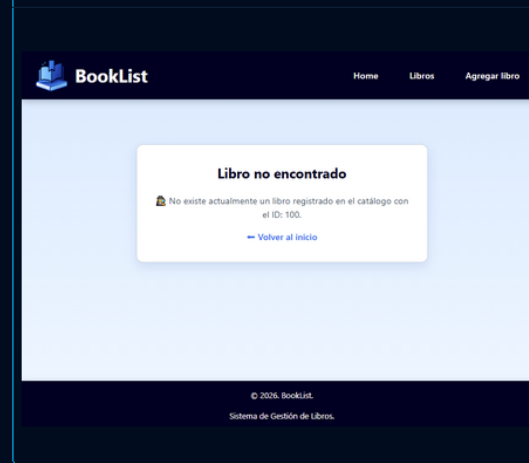
3. Invitado - Descripción oculta



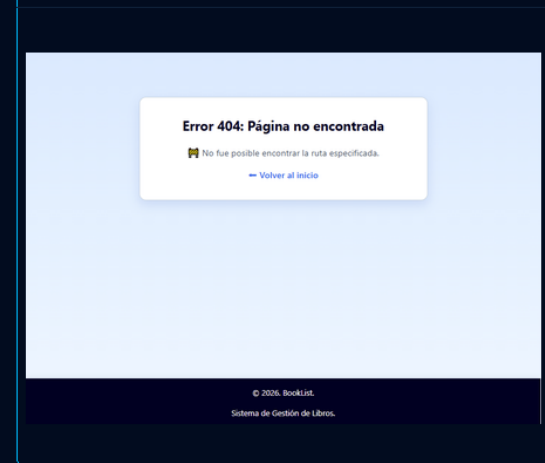
4. Invitado - Descripción mostrada



5. Libro no encontrado



6. Error 404 - Ruta inexistente



Maqueta de flujos IV

BookList - Flujo para agregar libros: usuario registrado e invitado

1. Usuario - Error por campo faltante

2. Usuario - Mensaje de ayuda

3. Usuario - Campos completados

4. Usuario - Libro agregado con éxito

5. Usuario - Nuevo libro en catálogo

6. Invitado - Error por campo faltante

7. Invitado - Mensaje de ayuda

8. Invitado - Campos completados

9. Invitado - Libro agregado con éxito

10. Invitado - Nuevo libro en catálogo